Алгоритми паралельного множення матриць

Інна Вячеславівна Стеценко
професор кафедри ІПІ НТУУ «КПІ ім. Ігоря Сікорського»,
д.т.н., професор

Послідовне множення матриць

$$\begin{pmatrix} a_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & a_{n-1,s-1} \end{pmatrix} \times \begin{pmatrix} b_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & b_{s-1,m-1} \end{pmatrix} = \begin{pmatrix} c_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & c_{n-1,m-1} \end{pmatrix}$$

$$c_{i,j} = \sum_{k=0}^{s-1} a_{i,k} \cdot b_{k,j}, \qquad 0 \le i < n, \qquad 0 \le j < m$$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Варіанти розпаралелення

1) Одночасно розраховувати окремі групи елементів результуючої матриці
2) Одночасно розраховувати окремі доданки елементів результуючої матриці

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Стрічкові алгоритми паралельного множення матриць

Перший алгоритм:
кожний процес розраховує рядок результуючої матриці,
кожна ітерація процесу – один елемент рядка результуючої матриці
між ітераціями відбувається циклічна пересилка стовпчиків другої матриці (тому і назва «стрічковий»)
У загальному випадку множення матриці А розміром $n \times s$ на матрицю В розміром $s \times m$ n процесів виконуватимуть m ітерацій обчислень за формулою:

$$\forall i - \text{процесу}, 0 \leq i < n, j - \text{ітерації}, j = i, \dots (i - 1 + m) \bmod m$$

$$c_{i,j} = \sum_{k=0}^{s-1} a_{i,k} \cdot b_{k,j}$$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Схема стрічкового алгоритму з обчисленням окремих елементів на кожній ітерації кожного процесу

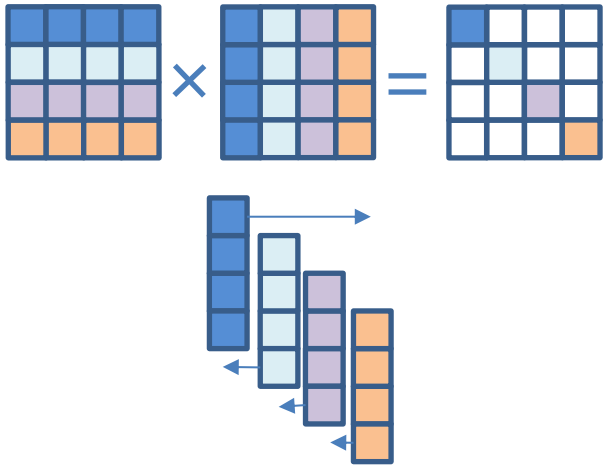
0-процес
1-процес
2-процес
3-процес

Кожний процес пам’ятає тільки один рядок матриці A та один стовпчик матриці B на кожній ітерації. Після кожної ітерації виконується циклічна передача стовпчиків матриці B між процесами

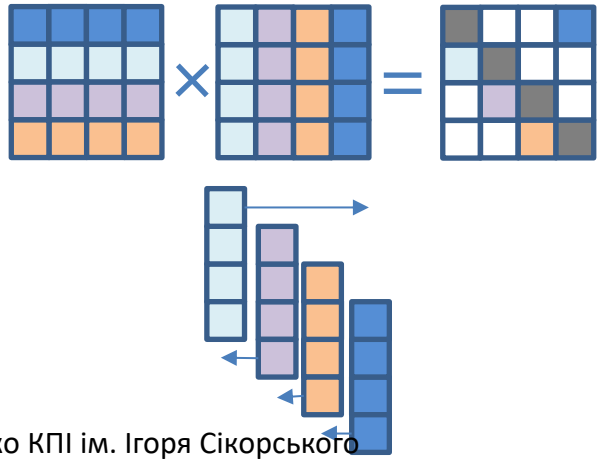
Обчислення елементу результуючої матриці не виконані

Обчислення елементу результуючої матриці виконані

0-ітерація



1-ітерація



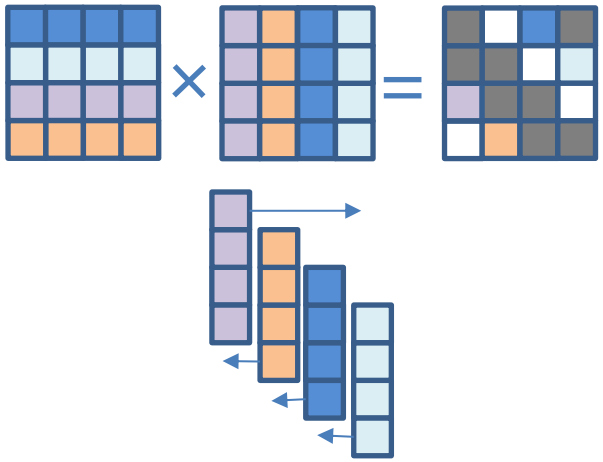
© І.В.Стеценко КПІ ім. Ігоря Сікорського

Схема стрічкового алгоритму з обчисленням окремих елементів на кожній ітерації кожного процесу

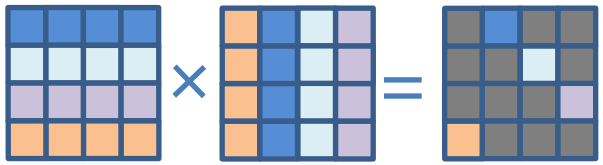
0-процес
1-процес
2-процес
3-процес

Кожний процес пам’ятає тільки один рядок матриці A та один стовпчик матриці B на кожній ітерації. Після кожної ітерації виконується циклічна передача стовпчиків матриці B між процесами

2-ітерація



3-ітерація



© І.В.Стеценко КПІ ім. Ігоря Сікорського

Ітерація	**Процес 0:** **0-рядок A**	**Процес 1:** **1-рядок A**	**Процес 2:** **2-рядок A**	**Процес 3:** **3-рядок A**
0	0-рядок A, **0 -стовпець B** $c_{00} = \sum a_{0k} b_{k0}$	1-рядок A, 1 -стовпець B $c_{11} = \sum a_{1k} b_{k1}$	2-рядок A, 2 -стовпець B $c_{22} = \sum a_{2k} b_{k2}$	3-рядок A, 3 -стовпець B $c_{33} = \sum a_{3k} b_{k3}$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Ітерація	**Процес 0: 0-рядок A**	**Процес 1: 1-рядок A**	**Процес 2: 2-рядок A**	**Процес 3: 3-рядок A**
0	0-рядок A, **0 -стовпець B** $c_{00} = \sum a_{0k} b_{k0}$	1-рядок A, 1 -стовпець B $c_{11} = \sum a_{1k} b_{k1}$	2-рядок A, 2 -стовпець B $c_{22} = \sum a_{2k} b_{k2}$	3-рядок A, 3 -стовпець B $c_{33} = \sum a_{3k} b_{k3}$
1	0-рядок A, 3 -стовпець B $c_{03} = \sum a_{0k} b_{k3}$	1-рядок A, **0 -стовпець B** $c_{10} = \sum a_{1k} b_{k0}$	2-рядок A, 1 -стовпець B $c_{21} = \sum a_{2k} b_{k1}$	3-рядок A, 2 -стовпець B $c_{32} = \sum a_{3k} b_{k2}$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Ітерація	Процес 0: 0-рядок A	Процес 1: 1-рядок A	Процес 2: 2-рядок A	Процес 3: 3-рядок A
0	0-рядок A, **0 -стовпець B** $c_{00} = \sum a_{0k} b_{k0}$	1-рядок A, 1 -стовпець B $c_{11} = \sum a_{1k} b_{k1}$	2-рядок A, 2 -стовпець B $c_{22} = \sum a_{2k} b_{k2}$	3-рядок A, 3 -стовпець B $c_{33} = \sum a_{3k} b_{k3}$
1	0-рядок A, 3 -стовпець B $c_{03} = \sum a_{0k} b_{k3}$	1-рядок A, **0 -стовпець B** $c_{10} = \sum a_{1k} b_{k0}$	2-рядок A, 1 -стовпець B $c_{21} = \sum a_{2k} b_{k1}$	3-рядок A, 2 -стовпець B $c_{32} = \sum a_{3k} b_{k2}$
2	0-рядок A, 2 -стовпець B $c_{02} = \sum a_{0k} b_{k2}$	1-рядок A, 3 -стовпець B $c_{13} = \sum a_{1k} b_{k3}$	2-рядок A, **0 -стовпець B** $c_{20} = \sum a_{2k} b_{k0}$	3-рядок A, 1 -стовпець B $c_{31} = \sum a_{3k} b_{k1}$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Ітерація	Процес 0: 0-рядок A	Процес 1: 1-рядок A	Процес 2: 2-рядок A	Процес 3: 3-рядок A
0	0-рядок A, **0 -стовпець B** $c_{00} = \sum a_{0k} b_{k0}$	1-рядок A, 1 -стовпець B $c_{11} = \sum a_{1k} b_{k1}$	2-рядок A, 2 -стовпець B $c_{22} = \sum a_{2k} b_{k2}$	3-рядок A, 3 -стовпець B $c_{33} = \sum a_{3k} b_{k3}$
1	0-рядок A, 3 -стовпець B $c_{03} = \sum a_{0k} b_{k3}$	1-рядок A, **0 -стовпець B** $c_{10} = \sum a_{1k} b_{k0}$	2-рядок A, 1 -стовпець B $c_{21} = \sum a_{2k} b_{k1}$	3-рядок A, 2 -стовпець B $c_{32} = \sum a_{3k} b_{k2}$
2	0-рядок A, 2 -стовпець B $c_{02} = \sum a_{0k} b_{k2}$	1-рядок A, 3 -стовпець B $c_{13} = \sum a_{1k} b_{k3}$	2-рядок A, **0 -стовпець B** $c_{20} = \sum a_{2k} b_{k0}$	3-рядок A, 1 -стовпець B $c_{31} = \sum a_{3k} b_{k1}$
3	0-рядок A, 1 -стовпець B $c_{01} = \sum a_{0k} b_{k1}$	1-рядок A, 2 -стовпець B $c_{12} = \sum a_{1k} b_{k2}$	2-рядок A, 3 -стовпець B $c_{23} = \sum a_{2k} b_{k3}$	3-рядок A, **0 -стовпець B** $c_{30} = \sum a_{3k} b_{k0}$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Стрічкові алгоритми паралельного множення матриць

Другий алгоритм:
кожний процес розраховує рядок результуючої матриці,
кожна ітерація додає до кожного елемента рядка результуючої матриці доданок
У загальному випадку множення матриці А розміром $n \times s$ на матрицю В розміром $s \times m$ n процесів виконуватимуть s ітерацій обчислень за формулою:

$$\forall i - \text{процесу}, 0 \leq i < n, \qquad k - \text{ітерації}, k = i, \dots (i - 1 + s) \bmod s$$

$$c_{i,j} = c_{i,j} + a_{i,k} \cdot b_{k,j}, \qquad j = \overline{0, m - 1}$$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

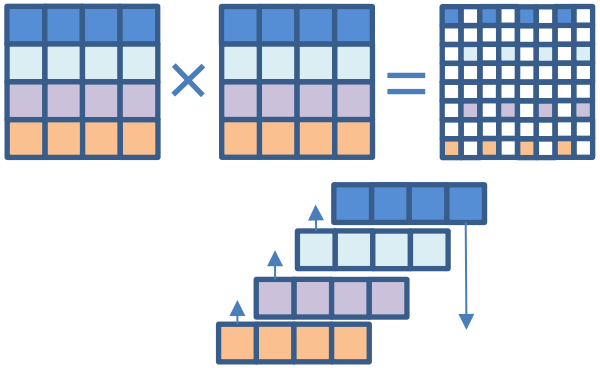
Стрічкові алгоритми паралельного множення матриць

0-процес
1-процес
2-процес
3-процес

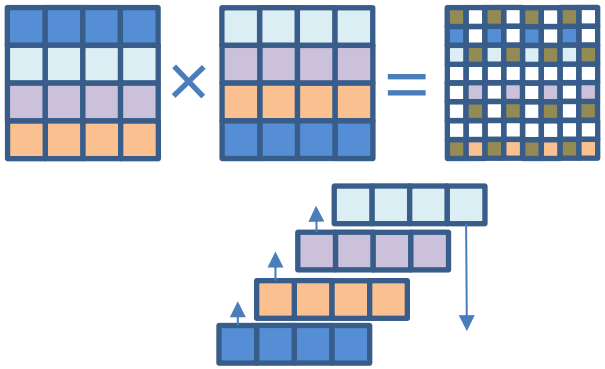
Кожний процес пам’ятає тільки один рядок матриці A та один рядок матриці B на кожній ітерації. Після кожної ітерації виконується циклічна передача рядків матриці B між процесами.

Чотири клітинки представляють чотири доданки одного елемента результуючої матриці. Якщо обчислення доданку ще не виконано, то клітинка без кольору, якщо виконано на попередній ітерації, то клітинка зафарбована в сірий колір, якщо триває обчислення, то зафарбована кольором відповідного процесу.

0-ітерація



1-ітерація



© І.В.Стеценко КПІ ім. Ігоря Сікорського

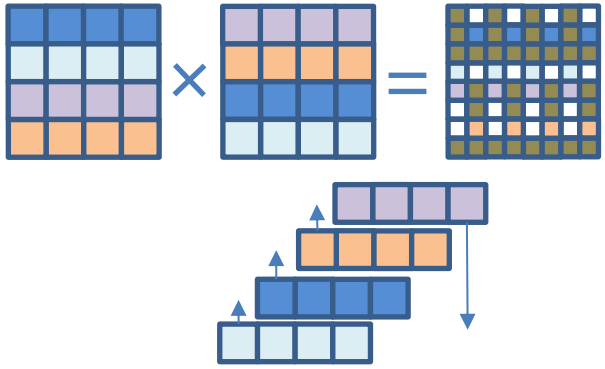
Стрічкові алгоритми паралельного множення матриць

0-процес
1-процес
2-процес
3-процес

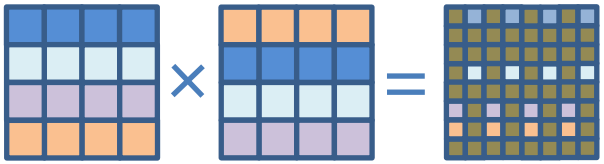
Кожний процес пам’ятає тільки один рядок матриці A та один рядок матриці B на кожній ітерації. Після кожної ітерації виконується циклічна передача рядків матриці B між процесами.

Чотири клітинки представляють чотири доданки одного елемента результуючої матриці. Якщо обчислення доданку ще не виконано, то клітинка без кольору, якщо виконано на попередній ітерації, то клітинка зафарбована в сірий колір, якщо триває обчислення, то зафарбована кольором відповідного процесу.

2-ітерація



3-ітерація



© І.В.Стеценко КПІ ім. Ігоря Сікорського

Ітерація	Процес 0: 0-рядок А	Процес 1: 1-рядок А	Процес 2: 2-рядок А	Процес 3: 3-рядок А
0	**0 -рядок В** $c_{0j} = a_{00}b_{0j}$, $j = 0,..m-1$	1 –рядок В $c_{1j} = a_{11}b_{1j}$, $j = 0,..m-1$	2 - рядок В $c_{2j} = a_{22}b_{2j}$, $j = 0,..m-1$	3 -рядок В $c_{3j} = a_{33}b_{3j}$, $j = 0,..m-1$
1	3 - рядок В $c_{0j} = c_{0j} + a_{03}b_{3j}$, $j = 0,..m-1$	**0 -рядок В** $c_{1j} = c_{1j} + a_{10}b_{0j}$, $j = 0,..m-1$	1 - рядок В $c_{2j} = c_{2j} + a_{21}b_{1j}$, $j = 0,..m-1$	2 -рядок В $c_{3j} = c_{3j} + a_{32}b_{2j}$, $j = 0,..m-1$
2	2 -рядок В $c_{0j} = c_{0j} + a_{02}b_{2j}$, $j = 0,..m-1$	3 - рядок В $c_{1j} = c_{1j} + a_{13}b_{3j}$, $j = 0,..m-1$	**0 -рядок В** $c_{2j} = c_{2j} + a_{20}b_{0j}$, $j = 0,..m-1$	1 -рядок В $c_{3j} = c_{3j} + a_{31}b_{1j}$, $j = 0,..n-1$
3	1 -рядок В $c_{0j} = c_{0j} + a_{01}b_{1j}$, $j = 0,..m-1$	2 - рядок В $c_{1j} = c_{1j} + a_{12}b_{2j}$, $j = 0,..m-1$	3 - рядок В $c_{2j} = c_{2j} + a_{23}b_{3j}$, $j = 0,..m-1$	**0 - рядок В** $c_{3j} = c_{3j} + a_{30}b_{0j}$, $j = 0,..m-1$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Експериментування

Ефективність паралельного алгоритму оцінюється прискоренням (speedup), що визначає у скільки разів зменшився час виконання алгоритму:

$$speedup = \frac{T_{sequential}}{T_{parallel}}$$

Matrix Size	Sequential Algorithm Time, s	Stripe Algorithm					
		4 threads		9 threads		25 threads	
		Time, s	Speedup	Time, s	Speedup	Time, s	Speedup
500							
1000							
1500							
2000							
2500							
3000							

Експериментування проводимо **обов'язково** при відключеному Інтернет-зв'язку та відсутності сторонніх обчислень, при підключенні до електромережі (для ноутбуку), з усередненням за 20 прогонами та, бажано, з розігрівом (warm up) JVM.

Обов'язково при наведенні результатів експериментів вказуємо технічні характеристики обладнання (кількість ядер, фізичних та логічних, частоту процесора) та операційну систему.

Наведення результатів експерименту у вигляді таблиці обов'язкове.

Візуалізація у вигляді графіків – бажано, але не обов'язково.

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Зауваження

- Для того, щоб зменшити кількість потоків, які потрібно створити, розрахунки у стрічкових алгоритмах виконують для груп рядків результуючої матриці.
- Для того, щоб зменшити очікування між ітераціями, зменшують кількість ітерацій. Для цього групують дії, що виконуються в ітераціях у звичайному алгоритмі.
- Кількість потоків для ефективної роботи алгоритму повинна бути близькою до кількості процесів, які можуть виконуватись паралельно обчислювальним ресурсом, на якому запускається алгоритм.

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Алгоритм Фокса

$$\begin{pmatrix} A_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & A_{q-1,q-1} \end{pmatrix} \times \begin{pmatrix} B_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & B_{q-1,q-1} \end{pmatrix} = \begin{pmatrix} C_{1,1} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & C_{q-1,q-1} \end{pmatrix}$$

$C_{ij} = \sum_{s=0}^{q-1} A_{is} B_{sj}$, де A_{is}, B_{sj} - блоки матриць *A* і *B* розміром $m \times m$

Ітерація
0
1
k
q-1

$$C_{ij} = \begin{aligned} & A_{i,i} B_{i,j} + \\ & + A_{i,(i+1)} B_{(i+1),j} + \ldots \\ & + A_{i,(i+k) \bmod q} B_{(i+k) \bmod q,j} + \ldots \\ & + A_{i,(i-1)} B_{(i-1),j} \end{aligned}$$

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Алгоритм Фокса

1. Спочатку виконується розсилка блоків A_{ii} та B_{ij} в усі процеси P_{ij}:

$$A_{ii}, B_{ij} \quad \rightarrow \quad P_{ij}$$

Знаходиться їх добуток і присвоюється результуючій матриці процесу

$$C_{ij} = A_{ii} B_{ij}$$

2. На кожній ітерації:
   - виконується циклічна пересилка блоків A (по другому індексу) і B (по першому індексу):

якщо i+1 == q, то i+1:=0,

$$A_{i(i+1)}, B_{(i+1)j} \rightarrow \quad P_{ij} \ .$$

   - знаходиться добуток блоків, які зберігаються в процесі, і додається до результуючої матриці процесу

Таким чином,

$$C_{ij} \mathrel{+}= A_{i,(i+k)\bmod q} \ B_{(i+k)\bmod q\,,j}$$, де *k* – номер ітерації

3. Результуючі матриці процесів передаються в головний процес, де з них складається результуюча матриця.

Якщо розмір матриць *A* і *B* $n \times n$, а кількість блоків по горизонталі і діагоналі q, то алгоритм реалізується на q^2 процесах, а розмір блоків матриць $m = n/q$.

Алгоритм Фокса.
Приклад для 4 процесів (q=2)

Крок 0

Розсилка в процеси блоків вихідних матриць

Ai**i** -> Pi**j**,

Bij -> Pi**j**.

Розрахунок результуючої матриці *Cij*

Ai**i** · Bij -> Ci**j**

A00 **B00** **C00 = A00 B00**	**A00** **B01** **C01 = A00 B01**
A11 B10 C10 = A11 B10	A11 B11 C11 = A11 B11

Крок 1

Розсилка в процеси наступних блоків вихідних матриць: **k** = **(i+1) mod 2**

Ai**k** -> Pi**j**,

B**k**j -> Pi**j**.

Розрахунок результуючої матриці

Cij+Ai**k**· B**k**j -> Ci**j**

A01 **B10** **C00 += A01 B10**	**A01** **B11** **C01 += A01 B11**
A10 B00 C10 += A10 B00	A10 B01 C11 += A10 B01

© І.В.Стеценко КПІ ім. Ігоря Сікорського

Експерименти

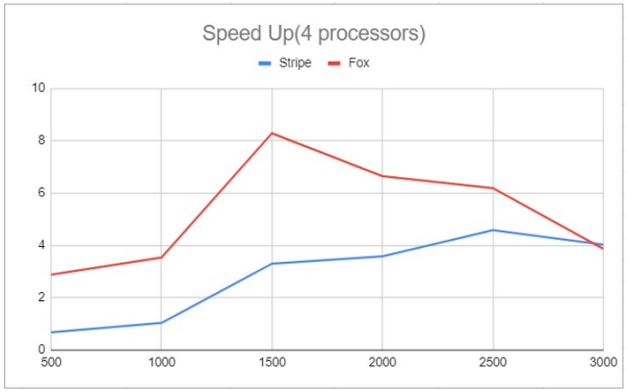
Matrix Size	Serial Algorithm Time, s	Fox Algorithm					
		4 threads		9 threads		25 threads	
		Time, s	Speedup	Time, s	Speedup	Time, s	Speedup
500							
1000							
1500							
2000							
2500							
3000							

Порівняйте з результатами, отриманими для стрічкового алгоритму.

Приклад порівняння

Matrix Size	Serial algorithm	Stripe algorithm					
		2 processors		4 processors		8 processors	
		Time	Speed Up	Time	Speed Up	Time	Speed Up
500	0,193	0,363	0,5316804408	0,282	0,6843971631	0,298	0,6476510067
1000	1,813	1,947	0,9311761685	1,749	1,036592338	1,824	0,9939692982
1500	19,087	6,289	3,034981714	5,774	3,305680637	5,688	3,355661041
2000	50,355	14,463	3,481642813	14,058	3,581946223	13,683	3,68011401
2500	116,588	27,174	4,290424671	25,435	4,583762532	23,983	4,861276738
3000	203,215	54,895	3,701885418	50,391	4,032763787	44,554	4,561094402

Matrix Size	Serial algorithm	Fox algorithm					
		2 processors		4 processors		8 processors	
		Time	Speed Up	Time	Speed Up	Time	Speed Up
500	0,193	0,1	1,93	0,067	2,880597015	0,092	2,097826087
1000	1,813	0,725	2,500689655	0,512	3,541015625	0,558	3,249103943
1500	19,087	3,566	5,352495794	2,302	8,291485665	2,182	8,747479377
2000	50,355	8,786	5,731277032	7,574	6,648402429	6,004	8,386908728
2500	116,588	25,357	4,597862523	18,826	6,192924679	13,505	8,632950759
3000	203,215	74,458	2,729256762	52,508	3,870172164	36,165	5,619106871



Інші паралельні алгоритми множення матриць

- Кеннона
- DNS (Dekel, Nassimi, and Sahni) алгоритм
- SUMMA (Scalable Universal Matrix Multiplication Algorithm)

Завдання до комп’ютерного практикуму «Розробка паралельних алгоритмів множення матриць та дослідження їх ефективності»

1. Реалізуйте стрічковий алгоритм множення матриць. **30 балів.**
2. Реалізуйте алгоритм Фокса множення матриць. **30 балів.**
3. Виконайте експерименти, варіюючи розмірність матриць, які перемножуються, для обох алгоритмів, та реєструючи час виконання алгоритму. Порівняйте отримані результати з часом виконання послідовного алгоритму. Визначить прискорення, отримуване в залежності від розміру матриць, які перемножуються. **20 балів.**
4. Виконайте експерименти, варіюючи кількість потоків, що використовується для паралельного множення матриць, та реєструючи час виконання. Порівняйте отримані результати з часом виконання послідовного алгоритму. Визначить прискорення, отримуване в залежності від розміру матриць, які перемножуються. **20 балів.**

!!! При проведенні експериментів дотримуйтесь таких умов:

- ✓ **у замір часу включати усі обчислення, необхідні для отримання результату, але не включати ініціалізацію матриць,**
- ✓ **позбавитись в алгоритмі усіх виведень на консоль**
- ✓ **відключений інтернет зв’язок,**
- ✓ **відключені усі сторонні програми (які можливо відключити),**
- ✓ **підключення до електромережі,**
- ✓ **розігрів на одному прогоні послідовного алгоритму,**
- ✓ **20 замірів часу для кожного результату,**
- ✓ **час послідовного замірювати після замірів часу паралельного алгоритму.**